

https://www.youtube.com/watch?v=68MnPCISVU4&list=PLLJKu7fP0xIxikCyG2D8_KOpabyfkRvuA

Danias vidoer om tråde

Teknologi

Tråde – del 1

Tråde

- **Dagens emner**
 - Sekventiel vs parallel
 - Hvad er processer og tråde?
 - Hvordan anvendes tråde
 - Problemer ved tråde

En Proces vs. En tråd (1/5)

- **Proces begrebet**

- Et OS skal håndtere en høj grad af nondeterminisme og dets logiske og fysiske parallelitet
- Begrebet en sekventiel proces introduceres derfor som en elegant måde at håndtere disse problemstillinger på
- En sekventiel proces (sometider kaldet en task) består af et program og et dataområde, begge i memory.
 - Samtidig har den en "thread of execution", repræsenteret af en "program counter" (PC) og en stack
- Nogle systemer tillader multiple threads of execution inde i en proces, hver med sin egen PC og stack

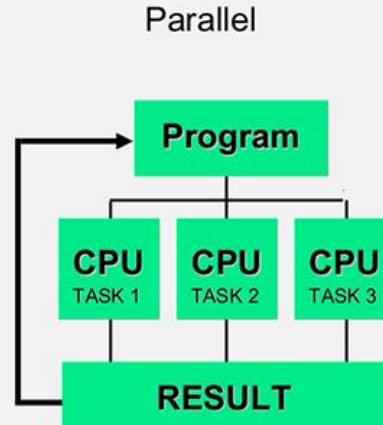
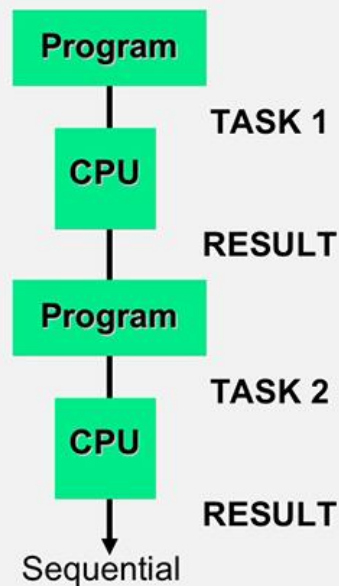
En Proces vs. En tråd (2/5)

- **Sekventiel vs parallel**
 - Konceptuelt har hver tråd sin egen CPU og memory
 - IRL er der et begrænset antal CPU'er og derfor er det OS opgave, at give de enkelte processer og tråde et timeslice på den enkelte CPU
 - En scheduling algoritme bestemmer hvor meget CPU tid de enkelte får

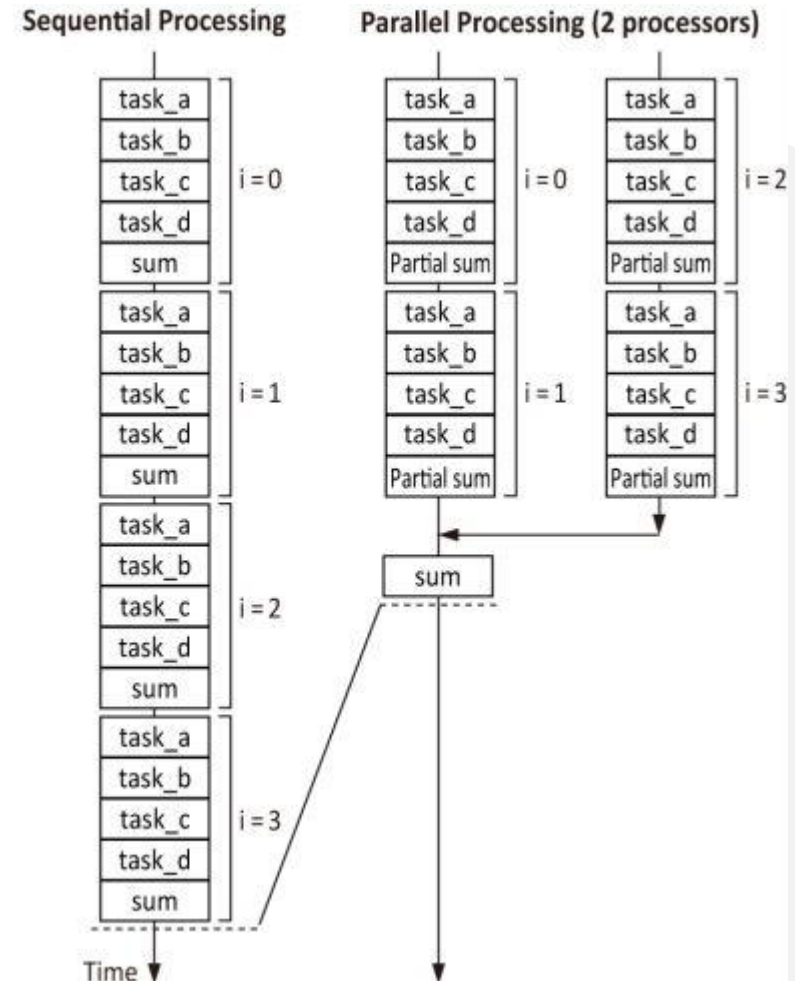
En Proces vs. En tråd (3/5)

- Sekventiel vs parallel

Sequential and parallel processing

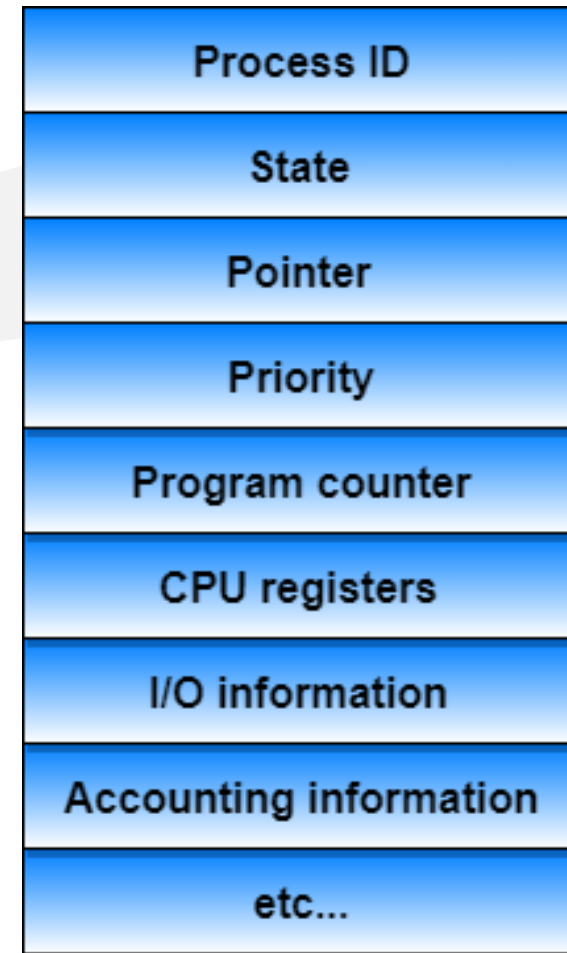


14



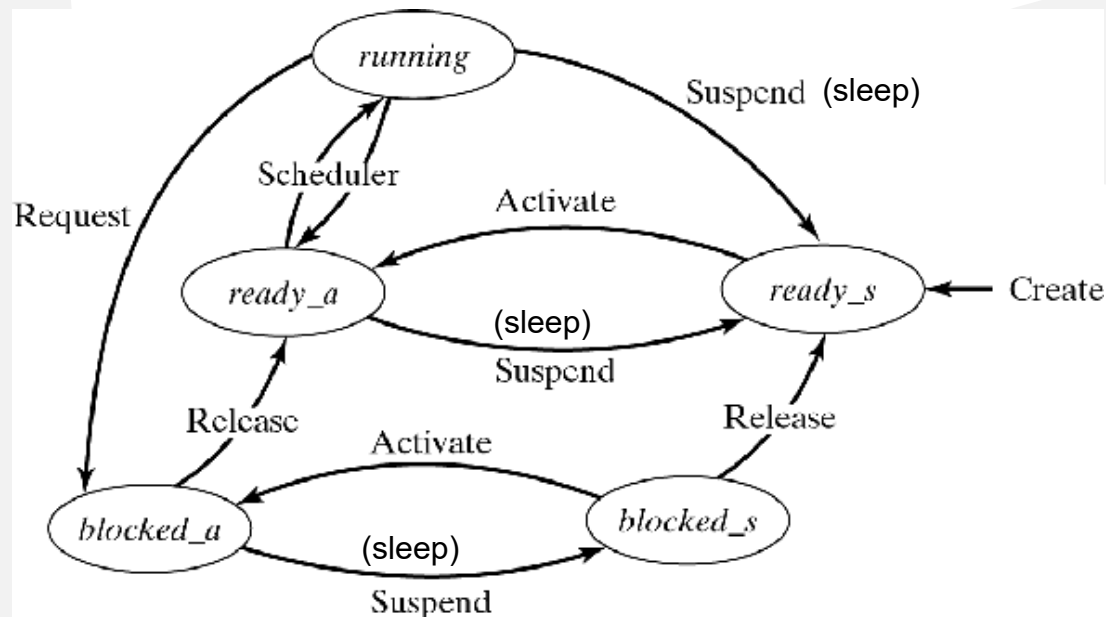
En Proces vs. En tråd (4/5)

- **Process descriptor** process control block
 - Aka. PCB(Process Control Block)
 - Oprettes af OS
 - Indeholder relevant info om processen
 - Fortæller om hvilken state/tilstand en proces er i
f.eks: new, ready, waiting, terminated



En Proces vs. En tråd (5/5)

- En proces kan være i forskellige states



running : Processen har tid på CPU'en og eksekverer instruktioner

ready_a : Processen er klar og aktiv, men venter på at OS planlægger(scheduling) den ind så den kan få CPU tid

ready_s : Processen er klar, men sover stadigvæk

blocked_s : Processen er blokeret (ikke aktiv) og tråden sover mens den venter på resursen

blocked_a : Processen er blokeret (men stadigvæk aktiv)

Tråde og programmering (1/2)

- Tråde giver mulighed for at køre dele af et program (proces) parallelt
 - Det kan f.eks. være at køre 2 eller flere metoder på samme tid
- Hvilke fordele giver dette ?
 - Mere responsivt GUI
 - Da brugergrænsefladen kan køre (reagere) samtidig med at der udføres andre beregninger
 - Gøres ved at der laves nye tråde til at udføre beregninger i baggrunden
 - Eks. I et skrive program laves der ofte backup af dokumenter, mens man kan skrive eller det at der køres stavekontrol samtidig
 - Dele løsningen / beregningen af en problemstilling op i flere mindre dele
 - En tråd pr. del
 - Optimering af CPU cykluser / resurser
 - Undgå at vente på langsomme resurser
 - Dele af et program der bruger langsomme resurser kører i deres egen tråd(e)
 - F.eks. Er lydkort (reelt tager det tiden der tager at afspille lyden) eller langtidslager (HDD) langsomme enheder

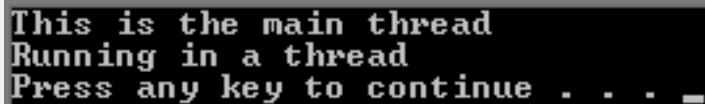
Tråde og programmering (2/2)

- **Et program har altid mindst en tråd**
 - Denne kaldes for "main" tråden – dvs. thread of execution
- **I udgangspunktet køre alt på main tråden**
 - Dvs. alt køre sekventielt og man kan regne med rækkefølgen af koden
- **Et program, der ikke har tråde eller andre teknikker til asynkront eller parallel afvikling, kan kun bruge en CPU kerne**
 - Note: Hvis intet program, som kørte på et OS, var multi-trådet ville udnyttelsen af en multi-core CPU derfor være begrænset af mængden af processor (programmer) der kørte

Tråde og C# (1/2)

- Thread klassen i C#

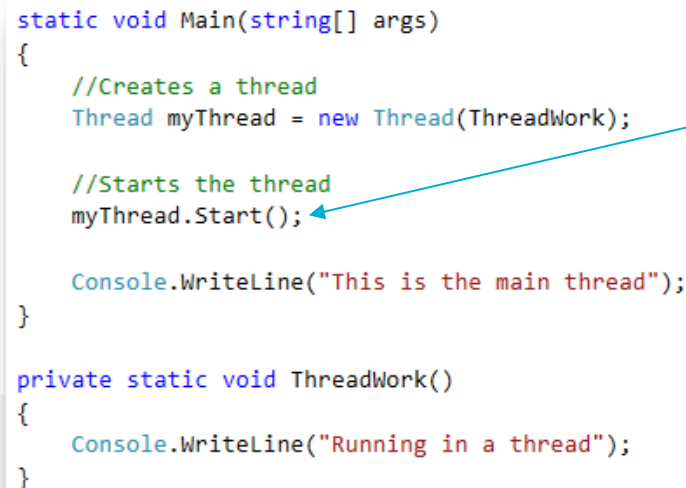
- Giver mulighed for at lave forgrunds- eller baggrundstråde
 - I udgangspunktet er tråde med Thread klassen forgrundstråde
- Klassen er overloaded til at acceptere en delegate parameter af typen
 - ThreadStart (en void metode uden argumenter)
 - ParameterizedThreadStart (en void metode, som tager et object som argument)



```
This is the main thread
Running in a thread
Press any key to continue . . . _
```

A screenshot of a console window with a black background and white text. The text shows the output of a C# program where the main thread prints 'This is the main thread' and 'Press any key to continue . . . _', and a newly created thread prints 'Running in a thread'.

Note: Outputtets rækkefølge kan ikke garanteres. Det er jeres OS, som bestemmer/ prioritere, hvilken tråd der kommer først. I dette eksempel er det main tråden, men det kunne lige så godt være den tråd der var i myThread.



```
static void Main(string[] args)
{
    //Creates a thread
    Thread myThread = new Thread(ThreadWork);

    //Starts the thread
    myThread.Start();

    Console.WriteLine("This is the main thread");
}

private static void ThreadWork()
{
    Console.WriteLine("Running in a thread");
}
```

A code snippet in C# showing how to create a new Thread object and start it. The Main method creates a Thread object named myThread with ThreadWork as the delegate, then calls myThread.Start(). The ThreadWork method is a private static void method that prints 'Running in a thread' to the console. A blue arrow points from the 'myThread.Start();' line in the code to the 'Note' on the right.

Note: Det er først når start kaldes at tråden kører metoden

Tråde og C# (2/2)

- **Tråde dør hvis de ikke holdes i live**
 - Hvis en metode, som anvendes af en tråd, afsluttes dør tråden
 - Man anvender generelt ikke døde tråde igen
 - I stedet for, sikre man at den ikke dør, dette gøres ved at metoden ikke afsluttes
- **Tråde kan holdes i live ved at koden i trådens metode kører indenfor en løkke struktur**
 - F.eks. En uendelig løkke : `while (true)`
 - Hint: en smart måde at sikre på, at man kan lukke en tråd korrekt ned kan være, at ikke at bruge "true" direkte, men bruge en boolsk field / attribute i stedet for
 - På den måde kan man ændre den til "false", når man vil bryde while løkken og dermed afslutter metoden og tråden dør

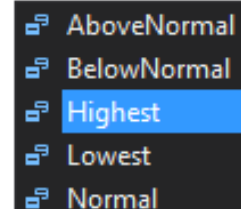
se evt øvelse 03 a

Tråde og CPU tid (1/2)

- Operativsystemet håndterer hvornår en tråd får lov til at køre på CPUen
 - Man kan godt have indflydelse på dette
 - Jeres OS planlægger tråde baseret på deres prioritet
 - Dvs. Tråde med høj prioritet får lov at køre på CPUen først / mest
 - En tråd stopper med at køre på CPUen hvis den venter på resurser
 - Det siges at den er blokeret og at den dermed sover
 - Man skal passe på med at tildele højere prioriteter til tråde, da det giver en mindre chance for at give andre tråde CPU tid
 - Disse prioriteter er: Highest > AboveNormal > Normal > BelowNormal > Lowest

Note: Det er ekstremt sjældent man bør ændre på trådes prioriteter. OS'et gør generelt et bedre job end man selv kan 😊

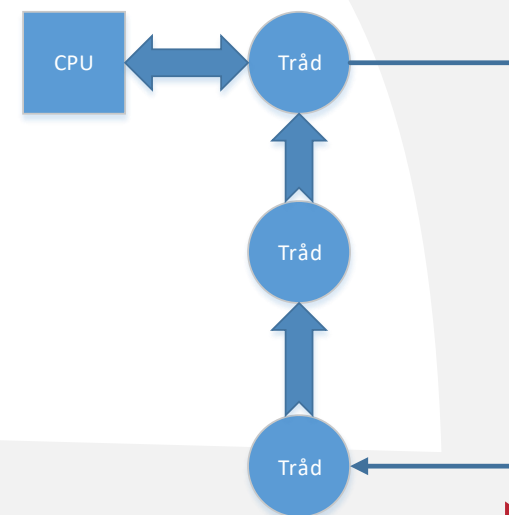
```
t1.Priority = ThreadPriority.Highest;  
t2.Priority = ThreadPriority.
```



AboveNormal
BelowNormal
Highest
Lowest
Normal

Tråde og CPU tid (2/2)

- Hvis tråden ikke selv frigiver CPUen sørger "thread scheduler" (OS) for at gøre det.
- En tråd har et "time quantum", hvilket vil sige at tråden kan bruge CPUen indtil tiden er gået
 - Så længe der ikke er en tråd med højere prioritet
 - Hvis flere tråde har samme prioritet bruges der et "round-robin" princip til at skifte tråde ud, dvs. på en fair måde
 - Prioriteten er dynamisk:
 - Tråde der venter på resurser får et priority boost



1. Simple tråde

- Lav en tråd der kalder en metode
- Metoden skal skrive en tekst ud til skærmen

2. Tråd der holdes i live

- Ændrer på øvelse 1 således tråden holdes i live
 - Dvs. at den ikke afsluttes automatisk
- Tråden skal altså blive ved med at skrive samme tekst ud til skærmen så længe programmet kører

3. Tråde skal afsluttes

- Ændre på øvelse 2 således tråden kan afsluttes korrekt, så det ønskes
- Hint 1: Brug hintet fra forrige slide
- Hint 2: Da I nu har 2 tråde, kan I stadigvæk modtage input i main tråden, således I f.eks. kan modtage input, som så kan bruges til at lukke tråde korrekt

Tråde og data (1/4)

øvelse 03 b

- Man kan give data videre til tråde på flere måder
 - Bruges generelt hvis man ønsker tråde arbejder med noget specifikt data
1. Man kan bruge ParameterizedThreadStart parameteret
 - Tager en void metode med et Object som argument

```
static void Main(string[] args)
{
    //Creates a parameter
    string message = "Hello World";

    //Instantiates the thread with thread with parameter function
    Thread t = new Thread(ThreadWithParameters);

    t.Start(message);
}

/// <summary>
/// A function with the correct structure
/// </summary>
/// <param name="o"></param>
private static void ThreadWithParameters(object o)
{
    string message = (string)o;

    Console.WriteLine("The thread received the following message: {0}", message);
}
```

The thread recived the following message: Hello World
Press any key to continue . . . _

Note: Her er data'et blot en string, men kunne sagtens være en egen defineret datatype f.eks. Car, Person eller Enemy

Tråde og data (2/4)

øvelse 03 c

2. Man henviser til den metode på en klasse der indkapsler data'et

```
class ThreadClass
{
    private string data;

    public ThreadClass(string data)
    {
        this.data = data;
    }

    public void ThreadWithParameter()
    {
        Console.WriteLine("The thread is using the following data: {0}", data);
    }
}
```

```
static void Main(string[] args)
{
    //Instantiates a ThreadClass
    ThreadClass threadClass = new ThreadClass("Hello World");

    //Instantiates a thread the uses the ThreadWithParameter method
    Thread t = new Thread(threadClass.ThreadWithParameter);

    //Starts the thread
    t.Start();
}
```

```
The thread is using the following data: Hello World
Press any key to continue . . . _
```


Tråde og data (3/4)

øvelse 03 d

3. Man kan bruge et lambda udtryk, som så igen kalder en metode med parameter

```
static void Main(string[] args)
{
    //Instantiates a thread by using a lambda expression
    Thread t = new Thread(() => ThreadWithParameters("Hello World"));

    //Starts the thread
    t.Start();
}

private static void ThreadWithParameters(string message)
{
    Console.WriteLine("The thread received the following message: {0}", message);
}
```

```
The thread recived the following message: Hello World
Press any key to continue . . . _
```

Læs mere om Lambda her: <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/operators/lambda-expressions>

Tråde og data (4/4)

øvelse 03 e

4. Man kan lave en klasse med data og en metode der så internt opretter tråden

```
class MyTask
{
    Thread internalThread;
    string data;

    public MyTask(string data)
    {
        this.data = data;
        internalThread = new Thread(ThreadMethod);
    }

    private void ThreadMethod()
    {
        Console.WriteLine("The thread received the following message : {0}", data);
    }

    public void Start()
    {
        internalThread.Start();
    }
}
```

```
static void Main(string[] args)
{
    MyTask myTask = new MyTask("Hello world");
    myTask.Start();
}
```

Note: Denne måde er særlig anvendelig, hvis man gerne vil have klassens ansvarsområde og funktionalitet håndteres således den selv har ansvaret på trådningen

F.eks. Hvis man har en selvstændigt og parallelt fungerende klasse – kunne være en Worker

```
The thread recived the following message: Hello World
Press any key to continue . . . _
```

Øvelser

4. Tråde med parametre

- Lav mindst 3 ud af de 4 forskellige metoder på forrige slides
- Note : I bør prioritere at få den sidste af de 4 muligheder med, da det er et pænere indkapslings koncept og ligger op til noget I kan bruge i jeres projekt periode

Tråde og programmets livscyklus

- En applikation bliver ved med at køre så længe at mindst 1 forgrundstråd kører

anbefaling trådene skal altid være baggrunds

- Main tråden er altid en forgrundstråd
 - Men lukkes når main metoden afslutter
- Husk at en tråd der laves via Thread klassen er som standard en forgrundstråd
 - Dette kan ændres via IsBackground
- Stavekontrollen i Word er f.eks. en baggrundstråd
 - Det ville ikke give mening, hvis den blev ved med at køre efter editoren er lukket ned

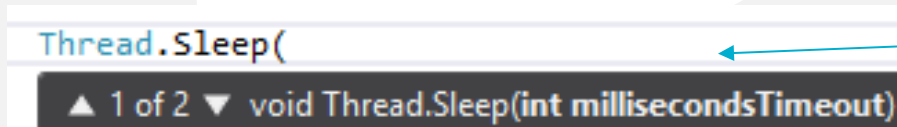
metoden som er IsBackground kører indtil at Main lukker

```
//Instantiates a thread by using a lambda expression
Thread t = new Thread(() => ThreadWithParameters("Hello World"));
t.IsBackground = true;
```

```
//Starts the thread
t.Start();
```

Tråde bruger ikke altid CPU resurser

- En ventende tråd bruger ikke CPU resurser
- Der kan være flere grunde til at en tråd venter
 - Via sleep instruktion, for at give CPU tid til andre tråde



```
Thread.Sleep(  
    ▲ 1 of 2 ▼ void Thread.Sleep(int millisecondsTimeout)
```

Note: Statisk metode på Thread klassen. Tråden vågner op igen når sleep tiden er gået

- Mens der ventes på I/O
- Mens der ventes på at en netværkspakke ankommer
- Mens der ventes på adgang til en synkroneringsmekanisme

Øvelser

øvelse 03 f

5. Korrekt nedlukning af et program

- Opret 2 metoder M1, M2
- Lad de 2 metoder eksekvere i hver sin tråd T1(M1), T2(M2)
- M1 skal indeholde et while loop, som skriver noget ud til jeres console
- M2 skal indeholde et for loop, som skriver noget ud til jeres console x antal gange.
- Sørg for at programmet lukker ned når T2 er færdig med at udføre sit arbejde
- Hint: Kig på baggrundstråde

Tråde er ikke uden problemer

- Ulemper ved tråde
 - Mere avanceret kode
 - Sværere at debugge, teste, og vedligeholde
 - Svært at holde tråde adskilt
 - Særlige tråd relaterede problemer kan opstå
 - Sværere for eksisterende kode at udnytte tråde, ofte skal koden skrives om

Tråd specifikke problemer

- **Hvad kan gå galt?**
 - Race conditions trådene konkurre med hinanden om at få ressourcer til at udføre dens kode
 - Dead locks
 - Live locks
 - Starvation den kommer ikke til at modtage nok ressourcer til at kunne køre
- **Hvorfor går det galt?**
 - Fordi trådene kan "pille" ved delte ressourcer
 - Ressurser kan her være data f.eks. i variabler og objekter
 - Fordi ikke alle operationer (eller kombinationer af operationer) er atomare(udelelige)

Race conditions (1/3)

```
//State er en delt resurse
public static int state = 5;

static void Main(string[] args)
{
    //Opretter og starter 2 tråde
    for (int i = 0; i < 2; i++)
    {
        Thread t = new Thread(RunMe);
        t.Start();
    }
}

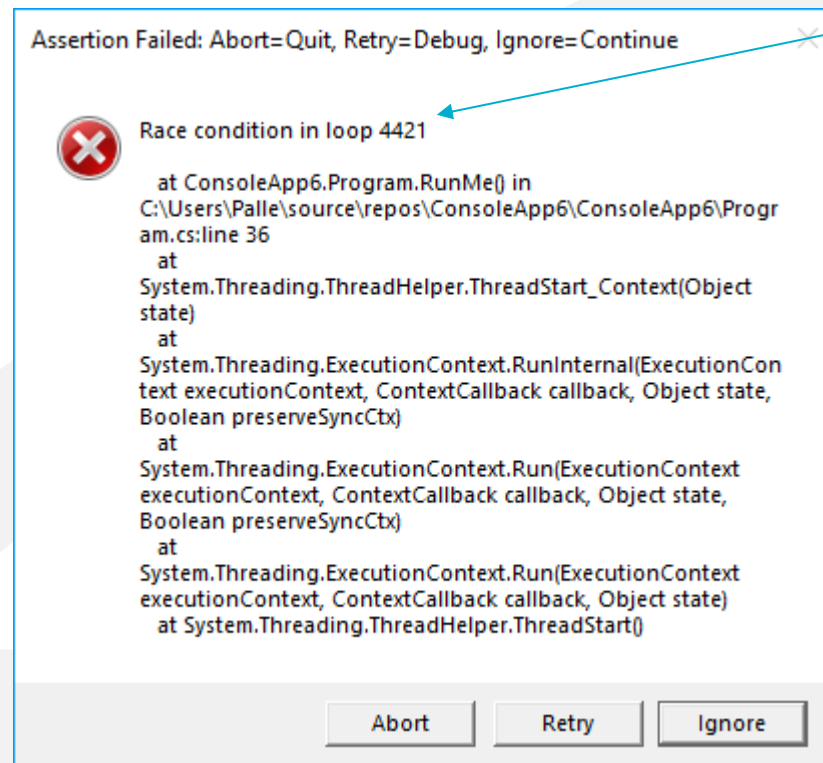
static void RunMe()
{
    int i = 0; //Anvendes til at finde ud af hvornår race condition opstår
    while (true)
    {
        if (state == 5) // Kontrollere om state er 5
        {
            state++; //Tæller state 1 op
            Trace.Assert(state == 6, "Race condition in loop " + i); //Tjekker om state er 6, hvis ikke = fejl
        }
        state = 5; // Nulstiller state til 5
        i++;
    }
}
```

man kan ikke være sikker på, at tråden for kørt if færdig, før den næste vil forsøge

fx tråd 1 kommer ind i if og ændre state til 6. Tråd 2 kan derfor ikke ind i if og ændre state tilbage til 5

Race conditions (2/3)

- Når vi kører vores program får vi følgende fejlbesked



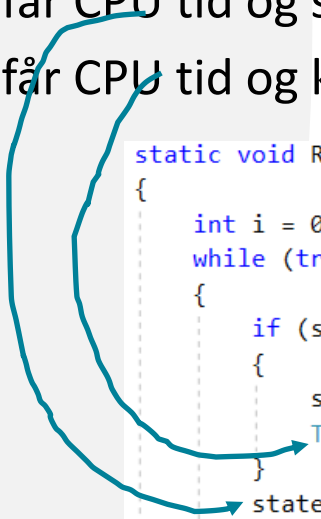
Bemærk : Der er ingen garanti for hvornår (hvilket loop) fejlen sker i. Dette påvirkes af OS'et schedulering af tråde og derfor af alle programmer der kører på computeren

Race conditions (3/3)

- Race conditions

- Dette sker pga. følgende:
- Tråd 1 får CPU tid og forøger værdien af state fra 5 til 6
- Tråd 2 får CPU tid og sætter værdien til 5
- Tråd 1 får CPU tid og kontrollerer at værdien er 6 hvilket er "false"

```
static void RunMe()
{
    int i = 0; //Anvendes til at finde ud af hvornår race condition opstår
    while (true)
    {
        if (state == 5) // Kontrollere om state er 5
        {
            state++; //Tæller state 1 op
            Trace.Assert(state == 6, "Race condition in loop " + i); //Tjekker om state er 6, hvis ikke = fejl
        }
        state = 5; // Nulstiller state til 5
        i++;
    }
}
```



Øvelser

øvelse 03 g

6. Fremprovoker en race condition

- Skriv koden fra forrige slides
- Sørg for at I får en forståelse for hvad der sker og hvorfor jeres racecondition opstår.

Petersons algoritme (1/2)

- Er en software løsning på problemet med en kritisk sektion formuleret af Gary L. Peterson i 1981
- Gør brug af 2 variabler

```
static bool[] flag = new bool[] { false, false };  
static int turn;
```

Flag bruges til at angive en tråd
ønsker adgang til den kritiske region

Angiver hvilken tråds tur der skal prioriteres

T0:

```
flag[0] = true;  
turn = 1;  
while (flag[1] == true && turn == 1)  
{  
    // busy wait  
}  
// critical region  
flag[0] = false;
```

Venter på T1 bliver færdig

T1:

```
flag[1] = true;  
turn = 0;  
while (flag[0] == true && turn == 0)  
{  
    // busy wait  
}  
// critical region  
flag[1] = false;
```

Venter på T0 bliver færdig

Note 1: T1 og T2 køre i parallel dvs. 2 separate tråde

Note 2: Algoritmen er tilpasset så den ligger tæt op af C#

Petersons algoritme (2/2)

- Selvom algoritmen er fancy, er det desværre ikke altid nemt at håndtere kritiske sektioner med den
- Den er svær at forstå og verificere. Særligt hvis vi har mange processer og delte ressourcer
- Den virker kun på processer der konkurrerer ikke på processer der samarbejder
- Den gør at processen der venter konstant skal teste om den fælles ressource er tilgængelig. Dette stjæler clock cycles
 - Dette kaldes også for busy waiting
- Den skalere ikke særligt godt, da man skal lave kode tilpasninger pr. tråd!
- Vi ønsker derfor en mere praktisk tilgang til processer/tråde
 - Hvilket vi ser på i morgen 😊

Øvelser

7. Prøv at implementere Petersons algoritme i C#, så en kritisk sektion beskyttes mod raceconditions i eksemplet på slide 25

– Hints :

- Kræver 2 unikke RunMe() metoder
- Læst evt. selv mere her :

https://en.wikipedia.org/wiki/Peterson%27s_algorithm